

Yggdrasil ERP — Platform Overview

Mimir Labs Technical Publication

Document Version: 1.0

March 2026

Classification: Public

Executive Summary

Yggdrasil ERP is a governance-native enterprise resource planning platform designed for manufacturing and B2B environments. It provides integrated management of customer relationships, sales operations, purchasing, manufacturing execution, warehouse logistics, financial accounting, project management, product lifecycle management, quality assurance, and service operations within a single multi-tenant system.

The platform is not a collection of loosely coupled services assembled around a database. It is a coordinated execution environment in which business state is governed by explicit rules, every meaningful transition is observable, and operational data remains trustworthy across organizational boundaries.

Yggdrasil is built by Mimir Labs as part of a broader data architecture stack that includes semantic discovery, data migration, live synchronization, and governance enforcement tools. This paper provides an architectural overview of the platform: its design philosophy, operational architecture, business modules, client interfaces, and relationship to the wider Mimir Labs ecosystem.

1. Design Philosophy

Enterprise software typically evolves by accretion. Modules are added, integrations are layered on, and operational discipline is delegated to the users who must navigate the resulting complexity. Over time, the system becomes a collection of capabilities without a coherent operational posture.

Yggdrasil takes a different approach. The platform is designed around five architectural principles that govern every layer of the system.

Deterministic execution. Business operations produce predictable outcomes. State transitions follow explicit rules enforced by a central state machine. Request handling, memory behavior, and service initialization are designed for consistency under sustained enterprise workloads.

Governed boundaries. Security, authorization, and tenant isolation are architectural properties, not application conventions. Every request passes through a centralized middleware pipeline that validates identity, resolves tenant context, and enforces permissions before business logic executes. Unrecognized or insufficiently described routes fail closed.

Observable operations. Every consequential mutation is captured in a structured audit trail and emitted into a real-time event stream. The system is designed not merely to process transactions, but to produce an operational history that supports compliance, integration, and post-hoc analysis.

Canonical semantics. The database schema is the authoritative definition of enterprise meaning. Application code, APIs, and integrations derive from it. External systems adapt to the canonical model, not the reverse.

Practical deployability. The system is designed to run in production with a small operational surface. Health endpoints, structured logging, graceful shutdown, and containerized deployment keep operational complexity proportional to business complexity.

These principles are not aspirational guidelines. They are enforced in code, tested in CI, and observable in production behavior.

2. Architecture Overview

Yggdrasil consists of four primary components that share a single canonical data model.

Server

The application server is implemented in C++17 on Qt 6. It provides RESTful HTTP and WebSocket interfaces, coordinates authentication, enforces authorization, manages state transitions, captures audit trails, and emits real-time events. The server supports both headless production deployment and operator-attended diagnostic modes.

The server registers over 400 API endpoints across 31 route modules. Every endpoint inherits the same security envelope: CORS handling, rate limiting, JWT validation, tenant resolution, and role-based access control. No endpoint operates outside this governed boundary.

Desktop Client

The desktop client is a native Qt 6 application built with C++17 and QML. It serves as the reference user interface implementation for the platform. All features are designed and validated in the desktop client before being reproduced in the web interface.

The desktop-first development model ensures that the platform remains suitable for operational environments where browser constraints are limiting: manufacturing floors, warehouses, service desks, and finance departments where operators interact with ERP systems continuously throughout the workday.

Web Application

The web application provides browser-based access using Next.js, React, and TypeScript. It communicates exclusively through the same public REST and WebSocket interfaces used by the desktop client, ensuring that the web interface cannot diverge from the platform's governed API surface.

The web application also exposes portal interfaces for account management, onboarding, documentation, and support interactions.

Database

The database layer is implemented on PostgreSQL. It contains 166 tables organized across 17 business domains, with row-level security enforcing tenant isolation at the database level. The schema uses UUID primary keys, PostgreSQL ENUM types for lifecycle states, JSONB columns for flexible structures, and a sequential migration system that evolves the schema under version control.

The schema is known as Mimisbrunnr, and it serves a dual purpose: as the operational persistence layer for Yggdrasil, and as the canonical semantic reference model consumed by every other tool in the Mimir Labs ecosystem.

3. Business Modules

Yggdrasil provides ten integrated business modules that cover the operational scope of a manufacturing and B2B enterprise.

Customer Relationship Management. Accounts, contacts, opportunities, leads, and pipeline management. CRM entities serve as the starting point for the order-to-cash flow, linking customer records to quotes, orders, and invoices downstream.

Sales. Quotes, sales orders, invoices, payments, and commission tracking. Sales operations are governed by the state machine, ensuring that documents progress through defined lifecycle stages and that financial implications are captured at each transition.

Purchasing. Purchase orders, suppliers, receipts, bills, and three-way matching. The purchasing module connects upstream to inventory requirements and downstream to accounts payable, creating a traceable procure-to-pay flow.

Manufacturing. Work orders, bills of materials, shop floor operations, routings, and OEE tracking. Manufacturing execution is tightly integrated with PLM for engineering data, warehouse for material

consumption, and quality for inspection and nonconformance reporting.

Warehouse. Inventory management, warehouse locations, stock transactions, picking, and cycle counts. The warehouse module maintains real-time inventory visibility across locations and supports directed putaway and picking operations.

Finance. General ledger, accounts receivable, accounts payable, banking, multi-currency accounting, and journal entries. Financial transactions are subject to state machine governance, ensuring that journal entries progress through draft, posted, and reconciled states under controlled conditions.

Projects. Project management, task tracking, time entries, budgets, and billing. The projects module supports both internal operations and billable client engagements, with time entries linked to financial and HR systems.

Product Lifecycle Management. Parts, engineering bills of materials, manufacturing bills of materials, engineering change requests, revisions, and materials management. PLM provides the engineering foundation that feeds manufacturing execution, connecting design intent to production reality.

Quality. 8D reports, CAPA actions, nonconformance reports, audit management, and inspection plans. Quality operations are integrated with manufacturing and service modules, enabling automatic NCR generation when inspection failures occur.

Service. Service tickets, RMA processing, warranty tracking, and maintenance orders. The service module supports both reactive incident management and proactive maintenance scheduling, with asset lifecycle tracking across customer deployments.

These modules are not independent applications. They share a common data model, a common state machine, a common audit infrastructure, and a common security envelope. Cross-module workflows such as order-to-cash, procure-to-pay, design-to-manufacture, and issue-to-resolution operate as continuous data flows rather than manual handoffs between disconnected systems.

4. Multi-Tenancy

Yggdrasil is designed for multi-tenant operation. Multiple organizations share a single application environment while maintaining strict data isolation.

Tenant isolation operates at multiple layers. At the application level, every API request is scoped to a tenant context derived from the authenticated session. At the database level, PostgreSQL row-level security policies ensure that queries cannot return records belonging to other tenants, even if application code fails to apply appropriate filtering. At the event level, real-time WebSocket channels and integration messages are scoped to tenant-specific streams.

This layered approach ensures that tenant boundaries remain intact even in the presence of application-level defects. A single programming error cannot compromise organizational data isolation because the database enforces its own independent boundary.

5. State Machine Governance

The state machine engine is the most architecturally significant service in the platform. It enforces valid lifecycle transitions across 23 entity types spanning all business modules.

Rather than allowing business objects to move through lifecycles by arbitrary field mutation, the state machine defines a transition graph for each governed entity type. Transitions that fall outside the allowed graph are rejected. When a valid transition occurs, the server performs the status update, audit capture, and event emission atomically within a single transaction.

This design ensures that the operational record, the audit history, and the real-time event stream remain synchronized. A state change is not merely a database update. It is a governed transition with traceable consequences.

The state machine covers sales quotes, orders, and invoices; purchase orders and bills; work orders and operations; PLM parts and engineering change requests; quality reports across all types; service tickets, RMAs, and maintenance orders; journal entries and financial documents; workflow instances and approval requests; form templates and submissions; and integration dead letters.

This breadth of coverage means that the lifecycle of virtually every business document in the system is explicitly governed rather than informally managed.

6. Security Architecture

Security in Yggdrasil is treated as an architectural property rather than an operational afterthought.

Authentication uses PBKDF2-based password hashing with multi-factor support through TOTP. Progressive account lockout prevents brute-force attacks. Session management uses short-lived access tokens and longer-lived refresh tokens delivered through secure HTTP-only cookies.

Authorization is enforced centrally through middleware that intercepts every API request. A role hierarchy ranging from system administrators to read-only users determines what actions each user may perform. Newly introduced endpoints automatically inherit the same security checks.

Input validation protects against injection attacks through parameterized queries, strict schema validation, and path traversal prevention. Sensitive configuration and credentials are encrypted at rest using authenticated encryption.

The platform maintains a comprehensive audit trail that captures every meaningful data mutation, lifecycle transition, and security-relevant event. This audit infrastructure supports both operational accountability and compliance requirements.

7. Real-Time Event Architecture

Yggdrasil includes a real-time event system that makes operational activity observable to clients, integration partners, and downstream processing systems.

The B2B Event Hub provides WebSocket-based event streaming with tenant-scoped channels. Events include state transitions, data changes, workflow activity, and system notifications. Events are emitted from governed application paths rather than from uncontrolled database triggers, ensuring that the event stream represents validated system behavior.

The Redpanda Relay bridges the internal event system to external Kafka-compatible message infrastructure. This enables event archival, cross-deployment synchronization, and integration with analytics pipelines and external systems.

Together, these components create an observable operational environment in which significant system activity is visible in real time without requiring polling or batch reconciliation.

8. The Mimir Labs Platform

Yggdrasil ERP does not exist in isolation. It is the operational execution layer within a broader data architecture stack developed by Mimir Labs.

Mimisbrunnr

The canonical semantic model. Mimisbrunnr is both Yggdrasil's production database schema and the universal reference vocabulary used by every tool in the platform. Its 166 tables across 17 domains define the shared meaning that enables semantic interoperability across the entire ecosystem.

Ratatosk

The semantic discovery tool. Ratatosk analyzes existing enterprise systems, classifies their data structures into business domains, and produces a machine-readable manifest describing the semantic meaning of enterprise data. This manifest becomes the foundation for migration, synchronization, and governance activities.

Ragnarok

The data migration engine. Ragnarok consumes Ratatosk manifests and executes deterministic, air-gapped data migrations between enterprise systems. It operates as a standalone desktop application with a staged wizard that enforces structure and traceability throughout the migration process.

Bifrost

The live data bridge. Bifrost maintains persistent synchronization between enterprise systems, supporting mirror, unidirectional, and bidirectional sync modes. It converts a one-time migration

outcome into a sustained operational state, ensuring that once systems are aligned, they remain aligned.

Jormungandr

The governance enforcement layer. Jormungandr imports canonical models produced by Ratatosk and continuously validates that enterprise data structures remain compliant. It converts one-time governance extraction engagements into long-term enforcement subscriptions.

Platform Lifecycle

These tools operate in a defined sequence that addresses the full lifecycle of enterprise data management.

Ratatosk discovers and formalizes meaning. Ragnarok migrates data into aligned structures. Yggdrasil operationalizes governed enterprise state. Bifrost maintains alignment across systems over time. Jormungandr enforces the canonical model that underlies the entire process.

Each tool is system-agnostic. While Yggdrasil is the default operational target, the semantic model defined by Mimisbrunnr can be applied to any compatible enterprise system. The tools communicate through structured manifests and canonical definitions rather than through direct coupling to a specific ERP implementation.

9. Deployment and Operations

Yggdrasil is designed for practical deployment in production environments.

The server supports both headless daemon operation for production and GUI-attended mode for development and diagnostics. Health endpoints report runtime and dependency status. Structured JSON logging with automatic rotation provides operational visibility. Graceful shutdown ensures that active requests, event buffers, and database connections are drained cleanly.

Production deployments use containerized orchestration with TLS termination at the network edge. The deployment model assumes an internet-shielded host with no application ports exposed directly to the public internet.

The CI/CD pipeline performs linting, type checking, unit testing, schema validation, and production build verification across all three application components before deployment. Server and client builds are compiled in Docker containers for reproducibility.

10. Why Governance-Native

The term governance-native describes a system in which governance properties are embedded in the architecture rather than bolted on after the fact.

Most enterprise systems evolve governance capabilities reactively. Audit logging is added when compliance requirements emerge. Access controls are tightened after security incidents. State validation is introduced when data quality degrades. These retroactive additions create a system that has governance features but is not governed by design.

Yggdrasil inverts this pattern. Tenant isolation is enforced at the database level from the first table. State transitions are governed by a central state machine from the first entity type. Audit logging captures every mutation from the first write operation. Authorization is enforced centrally from the first endpoint. Event emission is structured from the first status change.

This approach does not eliminate operational complexity. It makes operational complexity visible, traceable, and enforceable. The difference matters most in environments where data trustworthiness is not optional: manufacturing, finance, quality, and regulated industries where the cost of ungoverned operations is measured in compliance failures, audit findings, and operational breakdowns.

Conclusion

Yggdrasil ERP is not a feature checklist assembled into an application. It is a governed execution environment designed to maintain coherent enterprise state across operational domains, organizational boundaries, and system integrations.

Its architectural significance lies in the convergence of centralized security, explicit state governance, structured audit capture, observable event emission, and canonical data semantics into a single coordinated platform. These properties allow Yggdrasil to support not only transactional throughput but also the operational discipline that manufacturing and B2B enterprises require.

Within the broader Mimir Labs architecture, Yggdrasil serves as the point where canonical meaning becomes executable business state. It is the layer where governed semantics meet governed operations, and where the principles defined across the platform stack become operational reality.